

NAME: Shruthe

ROLL NO.: 241801265

Round Robin CPU Scheduling Algorithm

Write a C program to Implement the Round Robin (RR) CPU Scheduling Algorithm. The program should accept the number of processes, their Arrival Times (AT), Burst Times (BT), and a Time Quantum (TQ). The CPU should rotate through the processes in the ready queue, granting each a maximum of one TQ of execution time. Calculate the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for each process and the overall averages.

Input Format :

- The first line of input contains an integer representing the number of processes.
- The second line contains an integer representing the Time Quantum.
- The next lines contain two integers for each process, where
 - The first integer denotes the Arrival Time (AT)
 - The second integer denotes the Burst Time (BT) of the process.

Select Language: C (GCC 9.2.0)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, tq;
5
6     printf("Enter number of processes: ");
7     scanf("%d", &n);
8
9     printf("\nEnter Time Quantum: ");
10    scanf("%d", &tq);
11
12    int AT[n], BT[n], rem[n], CT[n], TAT[n], WT[n];
13
14    for(int i = 0; i < n; i++) {
15        printf("\nEnter AT and BT for P%d: ", i+1);
16        scanf("%d %d", &AT[i], &BT[i]);
17        rem[i] = BT[i];
18    }
19
20    int time = 0, completed = 0;
21
22    while(completed < n) {
23        int idle = 1;
24
25        for(int i = 0; i < n; i++) {
26            if(rem[i] > 0 && AT[i] <= time) {
27                idle = 0;
28
29                if(rem[i] > tq) {
30                    time += tq;
31                    rem[i] -= tq;
32                } else {
33                    time += rem[i];
34                    CT[i] = time;
35                    rem[i] = 0;
36                    completed++;
37                }
38            }
39        }
40
41        if(idle) {
42            time++;
43        }
44    }
45
46    float avgTAT = 0, avgWT = 0;
47
48    for(int i = 0; i < n; i++) {
49        TAT[i] = CT[i] - AT[i];
50        WT[i] = TAT[i] - rem[i];
51    }
52
53    for(int i = 0; i < n; i++) {
54        avgTAT += TAT[i];
55        avgWT += WT[i];
56    }
57
58    avgTAT /= n;
59    avgWT /= n;
60
61    printf("\nAverage TAT: %.2f\n", avgTAT);
62    printf("\nAverage WT: %.2f\n", avgWT);
63
64    return 0;
65 }
```

Select Language : C (GCC 9.2.0)

```
25     for(int i = 0; i < n; i++) {
26         if(rem[i] > 0 && AT[i] <= time) {
27             idle = 0;
28
29             if(rem[i] > tq) {
30                 time += tq;
31                 rem[i] -= tq;
32             } else {
33                 time += rem[i];
34                 CT[i] = time;
35                 rem[i] = 0;
36                 completed++;
37             }
38         }
39     }
40
41     if(idle) {
42         time++;
43     }
44 }
45
46 float avgTAT = 0, avgWT = 0;
47
48 for(int i = 0; i < n; i++) {
49     TAT[i] = CT[i] - AT[i];
50     WT[i] = TAT[i] - rem[i];
51 }
```

Custom Test Case

Custom Test

Input :

```
3
2
0.5
1.3
2.1
```

Expected Output :

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:

Average Turnaround Time: 6.33
Average Waiting Time: 3.33
```

Output :

```
Enter number of processes:
Enter Time Quantum:
Enter AT and BT for P1:
Enter AT and BT for P2:
Enter AT and BT for P3:
```

Select Language : C (GCC 9.2.0)

```
40
41     if(idle) {
42         time++;
43     }
44 }
45
46 float avgTAT = 0, avgWT = 0;
47
48 for(int i = 0; i < n; i++) {
49     TAT[i] = CT[i] - AT[i];
50     WT[i] = TAT[i] - BT[i];
51
52     avgTAT += TAT[i];
53     avgWT += WT[i];
54 }
55
56 avgTAT /= n;
57 avgWT /= n;
58
59 printf("\n\nAverage Turnaround Time: %.2f\n", avgTAT);
60 printf("Average Waiting Time: %.2f\n", avgWT);
61
62 return 0;
63 }
```